

[OOP]

Sistem Manajemen Perpustakaan Digital

Dokumentasi Implementasi OOP

1. Implementasi Relasi Antar Entitas
2. Operasi CRUD Lengkap
3. 4 Pilar OOP (Abstraksi, Enkapsulasi, Inheritance, Polimorfisme)
4. Exception Handling
5. Input Validasi

GitHub : github.com/fadhilyk/perpustakaan-digital

Bahasa : Python 3 + tkinter + JSON Storage

Pola : Model - Repository - Service - View

1. IMPLEMENTASI RELASI ANTAR ENTITAS

Relasi diimplementasikan in-memory via `_rekonstruksi_data()` pada kelas `Perpustakaan`. Setelah data dimuat dari JSON, referensi ID dikonversi menjadi Object Instance nyata.

Diagram Relasi Entitas:

Pengguna (ABC)

|-- Anggota [1:N] --> Peminjaman

+-- Petugas [1:N] --> Peminjaman

KategoriBuku [1:N] --> Buku [1:N] --> Peminjaman

Peminjaman menyimpan: id_buku, id_anggota, id_petugas (FK JSON)

File: `perpus_app/services/perpustakaan.py`

`perpus_app/services/perpustakaan.py`

```
45 | def _rekonstruksi_data(self) -> None:
46 |     # 1. Relasi Kategori <-> Buku
47 |     for buku in buku_list:
48 |         for kategori in kategori_list:
49 |             if buku.id_kategori == kategori.id_kategori:
50 |                 buku._kategori = kategori # ID -> Object
51 |                 kategori.tambah_buku(buku); break
52 |     # 2. Relasi Peminjaman <-> Buku & Anggota
53 |     for pinjam in peminjaman_list:
54 |         for buku in buku_list:
55 |             if pinjam.id_buku == buku.id_buku:
56 |                 pinjam._buku = buku; break
57 |         for anggota in anggota_list:
58 |             if pinjam.id_anggota == anggota.id_pengguna:
59 |                 pinjam._anggota = anggota; break
60 |     # 3. Relasi Anggota <-> List Peminjaman
61 |     for anggota in anggota_list:
62 |         anggota.set_list_peminjaman([p for p in peminjaman_list
63 |                                     if p.id_anggota == anggota.id_pengguna])
```

Proteksi Integritas Relasi:

`perpus_app/services/kategori_service.py`

```
62 | def hapus(self, id_kategori: int) -> None:
63 |     for buku in self._buku_service.get_all():
64 |         if buku.id_kategori == id_kategori:
65 |             raise KategoriMasihDipakaiError(
66 |                 f'Kategori masih digunakan buku "{buku.judul}"')
67 |     self.daftar_kategori = [k for k in self.daftar_kategori
68 |                             if k.id_kategori != id_kategori]
69 |     self._save_data()
```

2. OPERASI CRUD LENGKAP

Setiap entitas memiliki CRUD penuh di Service Layer, dipanggil dari View via Facade, disimpan ke file JSON via Repository.

Peta CRUD per Entitas:

Entitas	Create	Read	Update	Delete	File
Kategori	tambah()	get_all/get_by_id	update()	hapus()	kategori_service.py
Buku	tambah()	get_all/get_by_id	update()	hapus()	buku_service.py
Anggota	tambah()	get_all/get_by_id	update()	hapus()	anggota_service.py
Petugas	registrasi()	get_all/get_by_id	-	-	petugas_service.py
Peminjaman	pinjam()	get_all/get_aktif	-	kembalikan()	peminjaman_service.py

Contoh CRUD Buku — Service Layer:

perpus_app/services/buku_service.py

```
52 | # CREATE
53 | def tambah_buku(self, judul, pengarang, penerbit, tahun, stok, id_kategori):
54 |     new_id = generate_id(self.daftar_buku, 'id_buku')
55 |     buku = Buku(new_id, judul, pengarang, penerbit, tahun, stok, id_kategori)
56 |     self.daftar_buku.append(buku); self._save_data()
57 | # READ
58 | def get_all(self) -> list[Buku]: return self.daftar_buku
59 | # UPDATE
60 | def update(self, id_buku, judul, pengarang, penerbit, tahun, stok, id_kategori):
61 |     buku = self.get_by_id(id_buku) # raise jika tidak ada
62 |     buku.set_judul(judul); buku.set_stok(stok); self._save_data()
63 | # DELETE
64 | def hapus(self, id_buku):
65 |     self.daftar_buku.remove(self.get_by_id(id_buku)); self._save_data()
```

CRUD dari GUI — View Layer:

perpus_app/views/buku_frame.py

```
189 | def _on_tambah(self): self.facade.buku_service.tambah_buku(...); self._load_data()
190 | def _on_update(self): self.facade.buku_service.update(self.selected_id, ...); self._load_data()
191 | def _on_hapus(self): self.facade.buku_service.hapus(self.selected_id); self._load_data()
```

3. EMPAT PILAR OOP

3.1 ABSTRAKSI — Kelas Abstrak Pengguna

Pengguna adalah ABC dengan `@abstractmethod` tampilkan_info(). Tidak bisa diinstansiasi langsung.

perpus_app/models/pengguna.py

```
4 | from abc import ABC, abstractmethod
5 | class Pengguna(ABC):          # Kelas Abstrak
6 |     def __init__(self, id_pengguna, nama, no_telepon, email):
7 |         self._nama = ''; self.set_nama(nama)
8 |         self._email = ''; self.set_email(email)
9 |     @abstractmethod           # Wajib di-override subkelas
10 |     def tampilkan_info(self) -> str: pass
```

3.2 ENKAPSULASI — Atribut Private + Property Bervalidasi

Semua atribut pakai prefix `_` (private). Akses hanya via `@property`, mutasi via setter yang memvalidasi.

perpus_app/models/pengguna.py

```
14 |     @property
15 |     def nama(self) -> str: return self._nama # Getter
16 |     def set_nama(self, nama: str) -> None:   # Setter + validasi
17 |         valid, pesan = validasi_teks(nama, 'Nama')
18 |         if not valid: raise ValueError(pesan)
19 |         self._nama = str(nama).strip()
```

3.3 INHERITANCE — Anggota & Petugas mewarisi Pengguna

Kedua subkelas memanggil `super().__init__()` dan menambahkan atribut khasnya.

perpus_app/models/anggota.py

```
4 | class Anggota(Pengguna):      # Mewarisi Pengguna
5 |     def __init__(self, id_pengguna, nama, no_telepon, email, alamat):
6 |         super().__init__(id_pengguna, nama, no_telepon, email)
7 |         self._alamat = ''; self.set_alamat(alamat)
8 |         self._list_peminjaman = []
```

perpus_app/models/petugas.py

```
4 | class Petugas(Pengguna):      # Mewarisi Pengguna
5 |     def __init__(self, id_pengguna, nama, no_telepon, email, jabatan, username, pwd):
6 |         super().__init__(id_pengguna, nama, no_telepon, email)
7 |         self._jabatan = ''; self.set_jabatan(jabatan)
8 |         self._username = ''; self.set_username(username)
```

3.4 POLIMORFISME — Override tampilkan_info()

Method yang sama dipanggil pada tipe berbeda, menghasilkan output berbeda.

models/anggota.py vs models/petugas.py

```
18 | # Anggota - override #1:
19 | def tampilkan_info(self) -> str:
20 |     return f'Anggota: {self.nama}, Alamat: {self._alamat}'
21 | # Petugas - override #2 (berbeda):
22 | def tampilkan_info(self) -> str:
23 |     return f'Petugas: {self.nama} (Jabatan: {self._jabatan})'
24 | # Bukti Polimorfisme:
25 | for p in [anggota_obj, petugas_obj]: # keduanya list[Pengguna]
26 |     print(p.tampilkan_info())         # dipanggil sama, hasil beda
27 | # >> Anggota: Budi, Alamat: Jl. Merdeka 1
28 | # >> Petugas: fadhilyk (Jabatan: Administrator)
```

4. EXCEPTION HANDLING

Semua exception dalam hierarki kustom satu file. Setiap aksi GUI dibungkus try/except dua lapis.

4.1 Hierarki Exception Kustom:

perpus_app/exceptions/app_exceptions.py

```
1 | class AppError(Exception): pass # Base semua exception
2 | class ValidasiError(AppError): pass # Input tidak valid
3 | class DataTidakDitemukanError(AppError): pass # ID entitas tidak ada
4 | class StokTidakCukupError(AppError): pass # Stok buku habis
5 | class KategoriMasihDipakaiError(AppError): pass # Integritas relasi
6 | class AutentikasiError(AppError): pass # Login gagal
7 | class PenyimpananError(AppError): pass # Gagal baca/tulis JSON
```

4.2 Pola try/except Dua Lapis di View:

perpus_app/views/kategori_frame.py

```
130 | def _on_tambah(self):
131 |     try:
132 |         self.facade.kategori_service.tambah(nama, deskripsi)
133 |         MessageBox.show_info('Berhasil ditambahkan.', 'Berhasil')
134 |     except AppError as e:
135 |         MessageBox.show_error(str(e), 'Gagal Validasi') # Error bisnis
136 |     except Exception as e:
137 |         print(f'[Log] {e}') # Error tak terduga
138 |         MessageBox.show_error('Kesalahan internal.', 'Error')
```

4.3 Exception di Service Layer:

perpus_app/services/peminjaman_service.py

```
45 | def pinjam(self, id_anggota, id_buku, id_petugas):
46 |     buku = self.buku_service.get_by_id(id_buku) # raise DataTidakDitemukanError
47 |     if buku.stok <= 0:
48 |         raise StokTidakCukupError(f'Stok "{buku.judul}" habis.')
```

5. INPUT VALIDASI

Semua validasi terpusat di `utils/validators.py`. Dipanggil dari setter Model dan dari Service.

5.1 Daftar Fungsi Validator:

`perpus_app/utils/validators.py`

Fungsi	Aturan
<code>validasi_teks(teks, field)</code>	Tidak kosong, panjang 2-100 karakter
<code>validasi_email(email)</code>	Format: nama@domain.ext
<code>validasi_no_telepon(no_telp)</code>	Hanya angka, 8-15 digit
<code>validasi_tahun_terbit(tahun)</code>	Integer, antara 1400 - tahun sekarang
<code>validasi_stok(stok)</code>	Integer, tidak boleh negatif
<code>validasi_username(username)</code>	Alfanumerik, tidak boleh ada spasi
<code>validasi_password(pwd, konfirmasi)</code>	Min 8 kar, 1 kapital, 1 angka, 1 simbol
<code>validasi_pilihan(pilihan, field)</code>	Combobox tidak boleh kosong

5.2 Implementasi `validasi_password`:

```
30 | import re
31 | def validasi_password(password, konfirmasi) -> tuple[bool, str]:
32 |     if not password:         return False, 'Password tidak boleh kosong.'
33 |     if len(password) < 8:     return False, 'Password minimal 8 karakter.'
34 |     if not re.search(r'[A-Z]', password):
35 |         return False, 'Harus ada 1 huruf kapital (A-Z).'
36 |     if not re.search(r'[0-9]', password):
37 |         return False, 'Harus ada 1 angka (0-9).'
38 |     if not re.search(r'[!@#$%^&*()_+]', password):
39 |         return False, 'Harus ada 1 simbol (@, _, !, #, ...).'
40 |     if password != konfirmasi: return False, 'Password tidak cocok.'
41 |     return True, ''
```

5.3 Setter Model juga Memvalidasi:

`perpus_app/models/pengguna.py`

```
38 | def set_email(self, email: str) -> None:
39 |     valid, pesan = validasi_email(email)
40 |     if not valid: raise ValidationError(pesan)
41 |     self._email = str(email).strip()
```

5.4 Panduan Password di Form Register:

Di bawah field Konfirmasi Password muncul panduan:

Min. 8 karakter | Huruf kapital (A-Z) | Angka (0-9) | Simbol (@, _, !)

Pesan error muncul inline (merah) tanpa popup — pola SDD.md ss.6.2.

* RINGKASAN PEMETAAN FILE & FITUR

Pemetaan seluruh file per lapisan dan kaitannya dengan 5 poin presentasi.

Layer	File	Fitur / Poin OOP
Model	models/pengguna.py	Abstraksi (ABC), Enkapsulasi
Model	models/anggota.py	Inheritance, Polimorfisme
Model	models/petugas.py	Inheritance, Polimorfisme
Model	models/buku.py	Enkapsulasi, Relasi Kategori
Model	models/peminjaman.py	Enkapsulasi, Relasi 3 Entitas
Exception	exceptions/app_exceptions.py	6 Exception Kustom
Utils	utils/validators.py	8 Fungsi Validasi Terpusat
Repo	repositories/*.py	Persistensi JSON
Service	services/kategori_service.py	CRUD + Proteksi Relasi
Service	services/buku_service.py	CRUD Buku
Service	services/anggota_service.py	CRUD Anggota
Service	services/petugas_service.py	Auth, SHA-256
Service	services/peminjaman_service.py	CRUD Peminjaman, Cek Stok
Service	services/perpustakaan.py	Facade + Rekonstruksi Relasi
View	views/login_frame.py	Exception, Shake Animation
View	views/register_petugas_frame.py	Validasi, Exception
View	views/dashboard_frame.py	Statistik, Typewriter
View	views/kategori_frame.py	CRUD GUI + Exception
View	views/buku_frame.py	CRUD GUI + Cek Relasi
View	views/anggota_frame.py	CRUD GUI
View	views/peminjaman_frame.py	CRUD GUI, Kembalikan
View	views/laporan_frame.py	Laporan + Export CSV/Excel